

Week 06 Explained Student Worksheet

Inverted Index and Posting Lists

Name _____ Student ID _____ Group _____

Store only the document IDs associated with each term and retrieve matches by intersecting sorted posting lists. Compare this representation with the incidence matrix.

Use this worksheet during the two-hour practical session. Read, predict, run, and explain. The existing lab sheet contains the complete program; this companion explains key ideas and provides space to record your own evidence. Python 3.10 or later is sufficient; no pip packages are needed.

What you should be able to do

- Construct an inverted index with unique sorted posting lists.
- Calculate document frequency as posting-list length.
- Implement two-pointer posting-list intersection.
- Evaluate AND and NOT queries using postings.
- Compare postings and incidence matrices on the same collection.

Understand the concepts

An inverted index maps each term to the documents that contain it. The dictionary provides term lookup. A posting list is the ordered list of document IDs for one term. Document frequency $DF(t)$ is the number of distinct documents containing t , equal to the length of its posting list.

Following `section-2.py`, construction first stores document IDs in a set for each term. Repeated occurrences in one document therefore add no duplicate ID. After construction, each set becomes a sorted list. The sorting order used by the index and by intersection must agree.

Two-pointer intersection compares the current IDs in two sorted lists. If they match, output the ID and advance both pointers. Otherwise advance the pointer at the smaller ID. An exhausted list ends the scan. For list lengths m and n , the scan uses at most a linear number of comparisons, $O(m + n)$.

NOT selects the loaded IDs absent from a posting list; AND uses intersection. This implementation keeps the same limited query grammar as Week 5. The incidence matrix stores T times N bits as Python list values, while postings store only occurrences of terms in documents, plus dictionary/list overhead. Both express the same Boolean membership relation.

This week adds five original teaching documents to the five supplied examples. Weeks 6-10 reuse all ten files. IDs remain `doc1` through `doc10` for continuity. String sorting places `doc10` between `doc1` and `doc2`; this is intentional and is used consistently throughout the code.

Predict before running

For sorted postings `[doc1, doc3, doc4]` and `[doc2, doc3, doc4]`, predict the intersection before tracing pointers.

Prepare your terminal

Extract the course and open a terminal inside Week_06_Inverted_Index. Keep docs and data beside the Python files. The main demonstration is lab06_solution.py; your exercise starter is lab06_exercises.py. Starters run but deliberately contain unfinished TODO functions.

```
py -3.13 lab06_solution.py
py -3.13 lab06_exercises.py
```

Use py -3 or python instead if that is your working Python 3.10+ command.

Worked example with explanation

Run this complete two-pointer trace on two short lists. It isolates the retrieval operation from tokenization and index construction.

```
left = ["doc1", "doc3", "doc4"]
right = ["doc2", "doc3", "doc4"]
i = j = 0
result = []
while i < len(left) and j < len(right):
    if left[i] == right[j]:
        result.append(left[i])
        i += 1
        j += 1
    elif left[i] < right[j]:
        i += 1
    else:
        j += 1
print(result)
```

Expected output from an executed example

```
['doc3', 'doc4']
```

How each stage produces the result

1. The lists are sorted under the same string ordering. Each pointer starts at the first unread document ID.
2. If the IDs are equal, append the shared ID and advance both pointers. If they differ, advance only the pointer holding the smaller ID.
3. The smaller ID cannot match a later entry in the other sorted list. This is why advancing that pointer does not skip a possible match.
4. The loop stops when either list is exhausted. Each pointer moves only forward, so intersection takes work proportional to the combined list lengths.

Explain one intermediate value in your own words

Read the weekly program

The following excerpt is copied exactly from lab06_solution.py. It depends on the imports, constants, and helper functions in that file. Read it there rather than pasting it into an empty script and expecting the excerpt to run alone.

```
def intersect(list1, list2):
    result = []
    i = 0
    j = 0
    while i < len(list1) and j < len(list2):
        if list1[i] == list2[j]:
            result.append(list1[i])
            i += 1
            j += 1
        elif list1[i] < list2[j]:
            i += 1
        else:
            j += 1
    return result
```

Code walkthrough

1. The full build_index function accumulates document IDs in a set for each term, then sorts the set. Repetition within a document must not duplicate a posting.
2. intersect assumes both inputs are sorted and contain unique IDs. Its two bounds checks prevent reading beyond either list.
3. The three branches implement equality, left smaller, and right smaller. Inspect them with a pointer table instead of treating the loop as a black box.
4. The ten-document collection includes doc9, so some matches differ from Week 5. String sorting also places doc10 between doc1 and doc2; use the same ordering everywhere.

When left[i] is smaller than right[j], why is it safe to advance only i?

Connection to the course

Posting length provides document frequency for Week 7. The index records which documents contain a term; TF additionally records how often the term occurs within each document.

Guided practical activity

Work through the steps in order. Compare your predictions with observations and record at least one specific value per experiment. Keep original baseline data until you have saved the first transcript. Put experiments that change files in a disposable copy.

Step 1 Inspect all ten documents, noting which five have been added since Week 5.

Record the collection size and added documents. Changes in data can explain differences without any change in query logic.

Step 2 Use the unchanged tokenizer to accumulate a set of document IDs per term.

Add each document ID to a construction set. This removes duplicate postings while preserving the evidence that a term occurs.

Step 3 Sort every posting list and inspect python, java, machine, and data.

After sorting, compare the selected postings with the corresponding document texts. Posting length equals DF.

Step 4 Complete intersect in the starter and test empty, disjoint, and overlapping lists.

Trace i, j, current IDs, and result on several small inputs before testing the complete collection.

Step 5 Run the three Boolean demonstrations and inspect intermediate posting lists.

Inspect the postings printed for each query term and the complement universe for NOT.

Step 6 Build a Week 4-style matrix over the same ten documents and compare query results using both representations.

Build both representations from these same ten documents, then compare sets of matching IDs for identical queries.

Test record

Check to demonstrate	Actual observation	Pass or revise
Empty and disjoint lists return an empty list		
Overlapping sorted lists return each common ID once		
Postings queries agree with matrix queries on identical data		

Record one prediction that changed after running the program and explain why

Independent work and submission

Complete these selected tasks in your own exercise file. Use the existing lab sheet for additional exercises. Do not submit the unchanged starter or simply import a solution in place of implementing the requested function.

- Basic: print DF for python, machine, and data as the lengths of their postings.
- Implementation: complete two-pointer intersection without using set intersection as the algorithm.
- Debugging: repeat a term many times in a document. Verify that removing the construction set introduces duplicate postings.
- Interpretation: explain why the result for machine AND learning changes after adding doc9.

Diagnose common errors

An expected match is skipped. Check that both posting lists use the same sort order. The pointer algorithm relies on that ordering.

One document ID appears many times. Use a set during construction. Repeated terms increase TF later, not the number of postings for one document.

Submit these items

- Completed intersection code and tests for empty, disjoint, identical, and overlapping lists.
- Selected index entries, their document frequencies, and traces for the default queries.
- A short matrix/postings comparison using the same ten-document collection.

Your evidence should identify the input or query, expected result, actual result, and explanation. State any document edits or preprocessing changes so the result can be reproduced. Include this completed worksheet or a clearly numbered digital response.

Exit questions

What does an inverted index map from and to?

Why are sets used during construction?

Optional challenge

Implement sorted posting-list union and difference with two pointers. Test them independently before adding more query operators.

Source for this companion

Based on the supplied Week 06 lab sheet, shipped program, exercise starter, and sample data. The complete main program and expected_output.txt in this folder are the references for checking your baseline run.